

LKP End-to-End Regression

User Manual

Purpose	Compare LKP performance of a baseline kernel vs a patched kernel, on the host and inside KVM virtual machines, and record the results in one Excel workbook.
Workloads	hackbench, ebizzy, unixbench (LKP suite)

Contents

1. What the pipeline does
2. Prerequisites (one-time setup on the test machine)
3. How to start a run
4. Pipeline flow
5. Parameters reference (with screenshots)
6. Automatic checks performed by the pipeline
7. Paths used
8. Where to find the results
9. Troubleshooting

1. What the pipeline does

The pipeline runs the LKP benchmark suite on a **baseline (BASE)** kernel and a **patched (PATCH)** kernel on one bare-metal test machine (the Jenkins agent chosen by `NODE_LABEL`), and writes the numbers into an Excel workbook so the two kernels can be compared side by side.

Each LKP run is measured in one of three **test setups**:

Setup	Where LKP runs	BASE column	PATCH column
A. Machine only	On the host, no virtual machines	C	D
B. Machine + VMs as extra load	On the host, while per-OS VMs run alongside to add load	E	F
C. LKP inside the VMs	On the host and inside the VMs	G	H

The kernels can be **built by the pipeline** from a kernel git tree (`BUILD_BOTH`) or taken from kernels **already installed** on the machine (`MANUAL_BOTH_NO_BUILD`). In VM runs, the kernel under test is also installed inside the VM whose OS matches the host, so the guest runs the same kernel.

2. Prerequisites (one-time setup on the test machine)

These are **not** installed by the pipeline. Set them up once per test machine before the first run. The pipeline itself installs the kernel build toolchain, `python3-pip`, `pyfiglet / openpyxl`, `sshpas`, and clones/builds LKP.

The first parameter on the Build page, `PREREQUISITES_CONFIRMED`, repeats the file checklist:

☐ `PREREQUISITES_CONFIRMED`

BEFORE YOU RUN — confirm these one-time prerequisites on the SUT/agent (`NODE_LABEL`) and tick this box. The build fails in "Validate parameters" if this box is not ticked or if any of these are missing.

1) 12 golden VM qcow2 images must be in `/vms/jenkins_qcow2/*` (6 no-LKP + 6 LKP; one per OS: anolis/Rocky/Opencloud/Euler/Velinux/Ubuntu). If missing, copy them from the development host and fix ownership:

```
scp amd@10.86.26.102:/home/amd/vol1/images/jenkins/* /vms/jenkins_qcow2/  
chown qemu:qemu /vms/jenkins_qcow2/*
```

2) LKP result Excel template must be at `/vms/jenkins_excel_template/lkp_result_template.xlsx` (see `LKP_RESULT_TEMPLATE_PATH` below). If missing, copy it from the development host:

```
scp amd@10.86.26.102:/home/amd/vol1/images/jenkins_excel_template/lkp_result_template.xlsx  
/vms/jenkins_excel_template/
```

3) The Jenkins agent's remote root directory (e.g. `/tests/jenkins/workspace`) must be on a disk with enough space: LKP results and a copy of the Excel workbook are saved in the job workspace under `Run_<BUILD_NUMBER>/` and can be downloaded from the build's Workspace page.

2.1 Jenkins agent and access

Item	Requirement
Agent user	The Jenkins agent for the machine must run as root (the pipeline runs package installs, <code>grubby / grub</code> , <code>reboot</code> , <code>virsh</code> and <code>chown</code> without <code>sudo</code>).
Java	Java 21 (OpenJDK / Temurin 21) on the agent, required by Jenkins remoting.
Auto-reconnect	The agent must be online when the build starts (the build fails after 1 minute if no online agent has the <code>NODE_LABEL</code> label) and must reconnect by itself after a reboot (run it as a service). After each reboot the pipeline waits up to 1 hour for the node to return. On veLinux the machine can come back with a new IP after a kernel change: update the node's host IP in Jenkins (<i>Manage Jenkins</i> → <i>Nodes</i> → <i>the node</i> → <i>Configure</i>) and relaunch the agent within that hour, and the build continues.
Credential	A Jenkins <i>Username with password</i> credential for logging into the VMs; its ID goes in <code>VM_LOGIN_CREDENTIALS_ID</code> (default <code>vm-login</code>).

2.2 Virtualization (KVM / libvirt)

Item	Requirement / command
Packages	<code>qemu-kvm</code> , <code>libvirt</code> , <code>virt-install</code> , <code>qemu-img</code> RHEL family: <code>dnf install -y qemu-kvm libvirt virt-install</code> Debian family: <code>apt-get install -y qemu-kvm libvirt-daemon-system virtinst</code>

Hardware virtualization	VT/SVM enabled in BIOS; <code>/dev/kvm</code> present.
libvirt service	<code>systemctl enable --now libvirtd</code>
Default network	<code>virsh net-start default ; virsh net-autostart default</code> (VMs get their IP from its DHCP).
Kernel netfilter	The running host kernel needs the netfilter REJECT target (<code>CONFIG_IP_NF_TARGET_REJECT</code>), otherwise the default network cannot start.
Capacity	Each VM uses 33 GB RAM and 16 vCPUs. 6 VMs (+ <code>EXTRA_VMS</code>) must fit on the host.

2.3 Files and folders

Item	Location	If missing
12 golden VM images (6 no-LKP + 6 LKP, one per OS: anolis, Rocky, Opencloud, Euler, Velinux, Ubuntu)	<code>/vms/jenkins_qcow2/</code>	<pre>scp amd@10.86.26.102:/home/amd/vol1/images/jenkins/* /vms/jenkins_qcow2/ chown qemu:qemu /vms/jenkins_qcow2/*</pre>
LKP result Excel template	<code>/vms/jenkins_excel_template/lkp_result_template.xlsx</code>	<pre>scp amd@10.86.26.102:/home/amd/vol1/images/jenkins_excel_template/lkp_result_template.xlsx /vms/jenkins_excel_template/</pre>
Jenkins agent workspace (remote root directory)	<code>/tests/jenkins/workspace/</code>	Must be on a disk with enough space (<code>/tests</code> is a hardware disk: mount it, do not create it). LKP results and a copy of the Excel workbook are saved in the job workspace under <code>Run_<build number>/</code> .
Kernel git tree (only for <code>BUILD_BOTH</code>)	Path given in <code>EXISTING_REPO_PATH</code>	Clone the kernel tree on the machine first.

2.4 Network

The machine must reach **github.com** (LKP is cloned from `https://github.com/GirishP789/LKP_Automated.git`), the distro package repositories, and **pypi.org**.

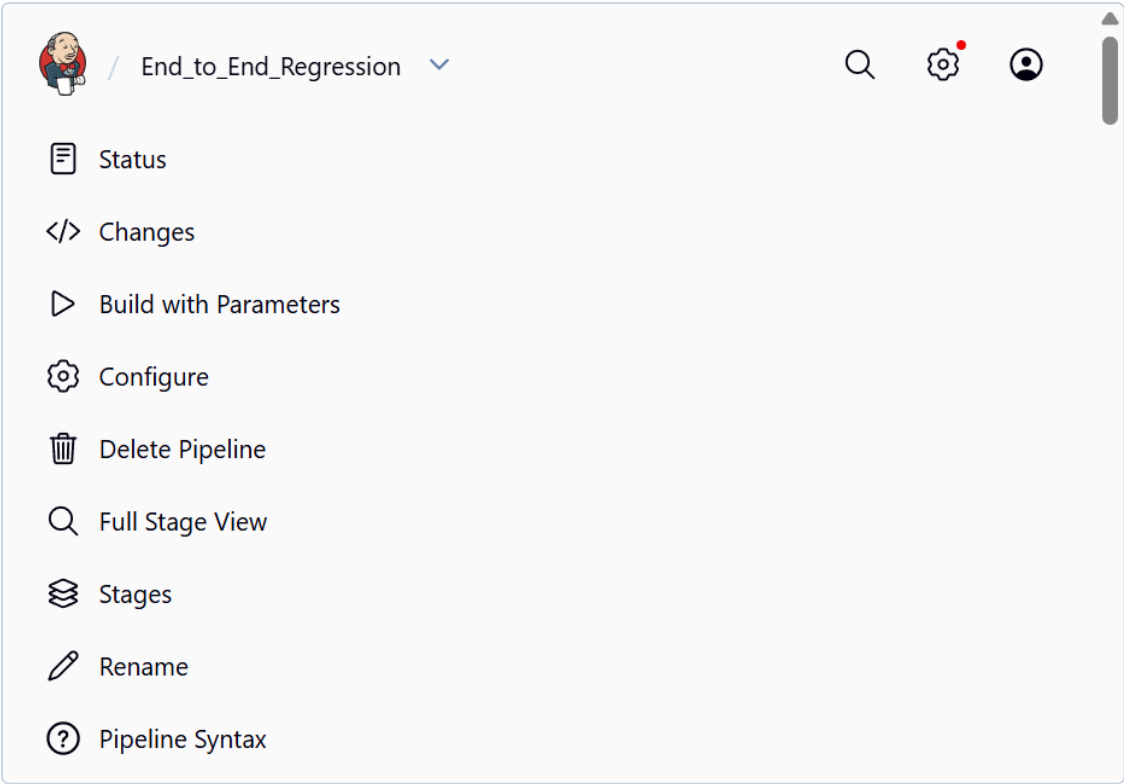
2.5 Golden image contents (already baked into the images)

The LKP images contain lkp-tests and `auto_lkp.service`, a running SSH server, and a sudo-capable login that matches `VM_LOGIN_CREDENTIALS_ID`.

A ready-to-paste AI prompt that checks and installs all of the above is in `Rollback_jenkins/LKP/prerequisites_installation.md`.

3. How to start a run

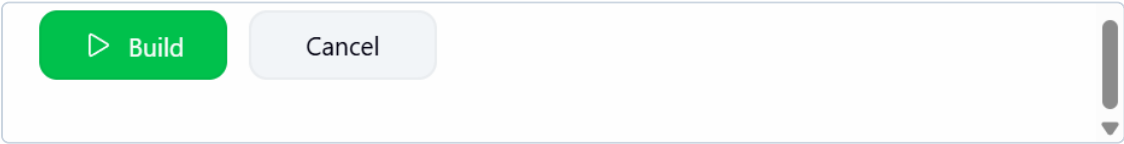
- 1. Open the LKP regression job in Jenkins and sign in.
- 2. Click **Build with Parameters** in the left menu.



- 3. The page lists all parameters (explained in section 5).



- 4. Fill in the parameters for the scenario you want. The minimum for any real run is: `NODE_LABEL` , `JIRA_ID` , `RPM_KERNEL_BUILD` (not NULL), the kernel inputs for that mode, and `LKP_RUN_OPTIONS` .
- 5. Click **Build** at the bottom of the page.



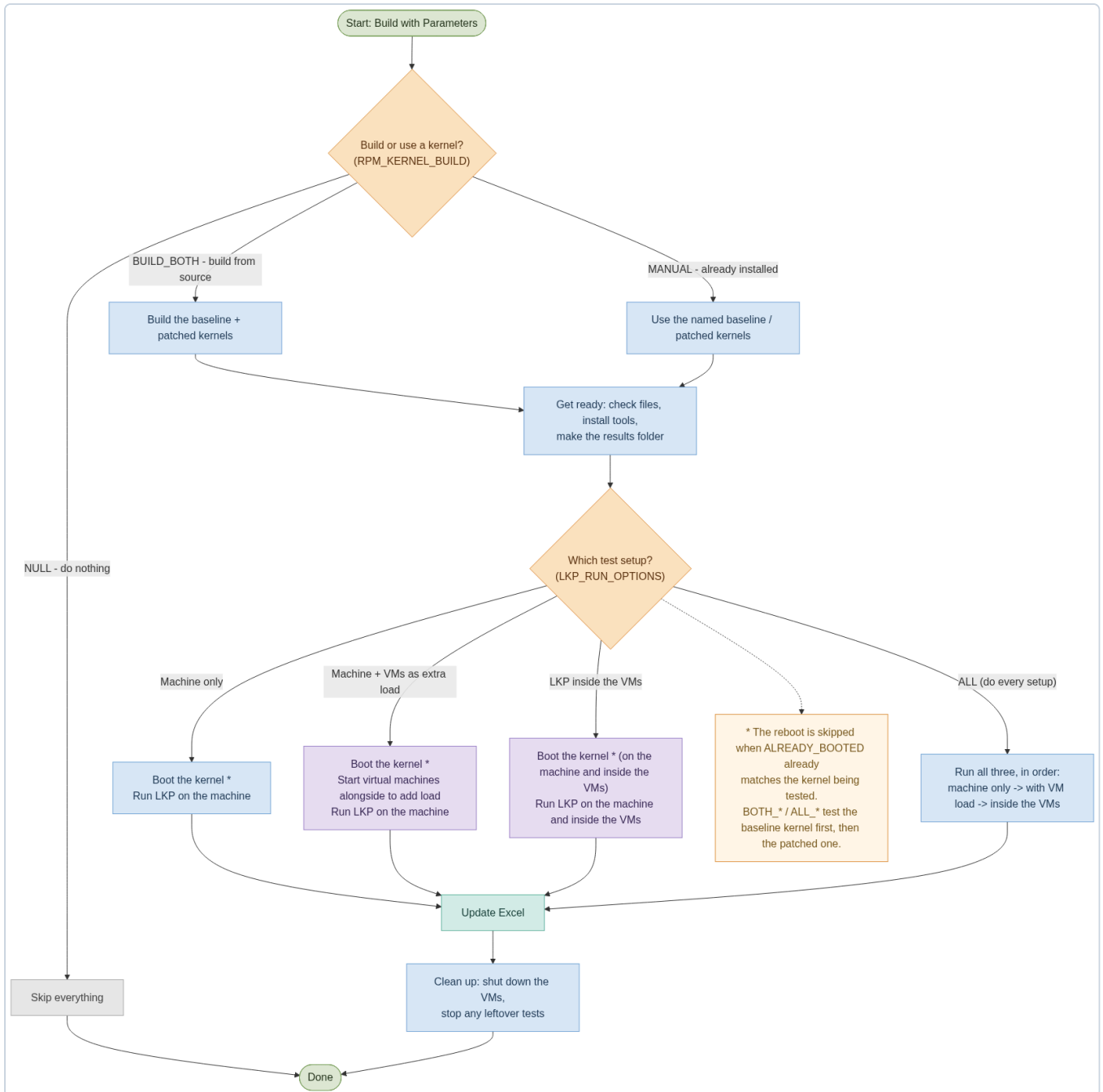
Quick recipes

Goal	Key parameters
Full comparison (all setups, both kernels), building from source	<code>RPM_KERNEL_BUILD=BUILD_BOTH</code> , <code>EXISTING_REPO_PATH</code> , <code>EXISTING_REPO_BRANCH</code> , <code>CHOOSE_BUILD_CONFIG</code> , <code>VM_CREATION_REQUIRED=YES</code> , <code>LKP_RUN_OPTIONS=ALL_RUNS</code>
Host-only comparison with kernels already installed	<code>RPM_KERNEL_BUILD=MANUAL_BOTH_NO_BUILD</code> , <code>MANUAL_BASE_KERNEL</code> , <code>MANUAL_PATCH_KERNEL</code> , <code>LKP_RUN_OPTIONS=BOTH_BASE_PATCH_LKP</code>

Re-run only the patched kernel inside VMs, using packages built earlier

RPM_KERNEL_BUILD=MANUAL_BOTH_NO_BUILD , MANUAL_PATCH_KERNEL ,
VM_CREATION_REQUIRED=YES , MANUAL_PATCH_RPM_PATH (or _DEB_PATH) from
/vms/jenkins_rpm/ , LKP_RUN_OPTIONS=PATCH_KERNEL_LKP_IN_VM

4. Pipeline flow



Stage order

#	Stage	What happens
1	Resolve node IP from agent	Waits at most 1 minute for an online agent with the <code>NODE_LABEL</code> label (the build fails otherwise), then records the machine's IP in the log.
2	Show LKP-running banner (motd)	Writes a green "LKP Regression is running in this system DON'T USE" box to <code>/etc/motd</code> (original saved to <code>/etc/motd.lkp_backup</code>), broadcasts "LKP Regression will start in 1 MINUTE" with <code>wall</code> , then waits 60 seconds.
3	Validate parameters	Fails fast on missing or wrong inputs (see section 6).
4	Prepare result backup dir	Creates <code><job workspace>/Run_<build number></code> inside the Jenkins job workspace (see section 8).

5	Detect OS / boot method	Chooses grubby (RHEL family) or grub (Debian family).
6	Install prerequisites	Build toolchain and Python helpers.
7	Checkout kernel repo / Build With Patch Kernel / Build Base Kernel	Only for <code>BUILD_BOTH</code> . Built packages are installed on the host and copied to <code>/vms/jenkins_rpm/</code> .
8	Resolve <code>PATCH_KERNEL</code> and <code>BASE_KERNEL</code>	Decides which kernel releases to boot.
9	Boot + LKP stages	Depends on <code>LKP_RUN_OPTIONS</code> : set default kernel → reboot (waits up to 1 hour for the agent to return) → verify kernel → LKP run(s). VM runs create the VMs, install the kernel in the matching guest, run LKP, then delete the VMs. <code>BOTH</code> / <code>ALL</code> options do the <code>BASE</code> kernel first, then <code>PATCH</code> .
10	Remove LKP-running banner (motd)	Restores the original <code>/etc/motd</code> .
-	Post (always)	Shows VM status, deletes any leftover stress VMs, kills leftover LKP processes, and restores <code>/etc/motd</code> even if the build failed or was aborted. Skipped when the agent was never reached.

With `RPM_KERNEL_BUILD=NULL_NO_KERNEL_WORKFLOW` every stage above except step 1 is skipped (without waiting for the agent), and nothing is cleaned up because nothing was created.

Run choices (`LKP_RUN_OPTIONS`)

Option	Kernel(s)	What runs	Excel columns
<code>BASE_KERNEL_AND_LKP</code>	baseline	LKP on the machine	C
<code>PATCH_KERNEL_LKP</code>	patched	LKP on the machine	D
<code>BOTH_BASE_PATCH_LKP</code>	baseline then patched	LKP on the machine	C, D
<code>BASE_KERNEL_NOLKP_IN_VM</code>	baseline	VMs as load, LKP on the machine	E
<code>PATCH_KERNEL_NOLKP_IN_VM</code>	patched	VMs as load, LKP on the machine	F
<code>BOTH_KERNELS_NOLKP_IN_VM</code>	baseline then patched	VMs as load	E, F
<code>BASE_KERNEL_LKP_IN_VM</code>	baseline	LKP on the machine and inside the VMs	G
<code>PATCH_KERNEL_LKP_IN_VM</code>	patched	LKP on the machine and inside the VMs	H
<code>BOTH_KERNELS_LKP_IN_VM</code>	baseline then patched	LKP on machine + VMs	G, H
<code>ALL_BASE</code>	baseline	All 3 setups	C, E, G
<code>ALL_PATCH</code>	patched	All 3 setups	D, F, H
<code>ALL_RUNS</code>	baseline then patched	All 3 setups, both kernels	C to H

5. Parameters reference

Parameters are listed in the order they appear on the Build page. Screenshots are taken from the live job.

5.1 General

PREREQUISITES_CONFIRMED checkbox

Default	unticked
Used when	Every run except NULL_NO_KERNEL_WORKFLOW (required)

Shows the one-time setup checklist (section 2). It **must be ticked**: the build fails in *Validate parameters* if it is left unticked. Tick it after checking the images, the template and the agent workspace disk. Screenshot in section 2.

NODE_LABEL string

NODE_LABEL

Jenkins agent label of the SUT. SUT IP is auto-resolved from the agent via `'hostname -I'`.

9dee

Default	9dee
Used when	Always

Jenkins agent label of the test machine. Every stage runs on this agent. Must not be empty.

JIRA_ID string required

JIRA_ID

Jira ticket number (e.g. 5748) used in the LKP result workbook name
DEAE_<JIRA_ID>_<OS>_<kernel>_<pipe>.xlsx.

Default	(empty)
Used when	Every run except NULL_NO_KERNEL_WORKFLOW

Jira ticket number. It names the result folder `/home/amd/DEAE_<JIRA_ID>/` and the workbook `DEAE_<JIRA_ID>_<OS>_<kernel>_<job>.xlsx`. The build fails in *Validate parameters* if it is empty.

LKP_RESULT_TEMPLATE_PATH string

LKP_RESULT_TEMPLATE_PATH

Path (on the agent) to the LKP result Excel template. Copied to /home/amd/DEAE_<JIRA_ID>/DEAE_<JIRA_ID>_<OS>_<kernel>_<pipe>.xlsx on first use, then edited in place (a copy is also saved in the workspace under Run_<BUILD_NUMBER>/).

/vms/jenkins_excel_template/lkp_result_template.xlsx

Default	/vms/jenkins_excel_template/lkp_result_template.xlsx
Used when	Every run except NULL_NO_KERNEL_WORKFLOW (checked in Validate parameters, used by every LKP report step)

Excel template on the machine. On the first LKP run of a JIRA/OS/kernel/job combination it is copied into /home/amd/DEAE_<JIRA_ID>/ ; later runs edit that copy in place, and after every LKP run a copy of the workbook is also saved in the job workspace under Run_<build number>/ . The build fails in *Validate parameters* if the template does not exist.

5.2 Kernel source

RPM_KERNEL_BUILD choice

RPM_KERNEL_BUILD

Kernel source (default NULL does nothing):
NULL_NO_KERNEL_WORKFLOW — skip build/install/boot/LKP.
BUILD_BOTH — build patch + base from EXISTING_REPO_PATH; kernel names discovered from install.
MANUAL_BOTH_NO_BUILD — no build; use preinstalled kernels named in MANUAL_*_KERNEL.

NULL_NO_KERNEL_WORKFLOW

Default	NULL_NO_KERNEL_WORKFLOW
---------	-------------------------

Choice	Meaning
NULL_NO_KERNEL_WORKFLOW	Do nothing (no build, boot, LKP, report or cleanup).
BUILD_BOTH	Build the patched and baseline kernels from EXISTING_REPO_PATH , install them, and copy the packages to /vms/jenkins_rpm/ .
MANUAL_BOTH_NO_BUILD	No build; use kernels already installed on the machine, named in MANUAL_BASE_KERNEL / MANUAL_PATCH_KERNEL .

5.3 Kernel build (only for RPM_KERNEL_BUILD=BUILD_BOTH)

EXISTING_REPO_PATH string

EXISTING_REPO_PATH

Required for BUILD_BOTH. Absolute path to the kernel git checkout on the agent (env.DIR).

Default	(empty)
Used when	BUILD_BOTH (required)

Absolute path to the kernel git checkout on the machine. The pipeline does not clone it; the path must exist and be a git working tree.

EXISTING_REPO_BRANCH string

EXISTING_REPO_BRANCH

Required when BUILD_BOTH. Branch to checkout in EXISTING_REPO_PATH before build.

Default	(empty)
Used when	BUILD_BOTH (required)

Branch checked out in EXISTING_REPO_PATH before building. The patched kernel is built from this branch.

BASE_HEAD_SHA_ID string

BASE_HEAD_SHA_ID

Optional (BUILD_BOTH only). Commit SHA (7-40 hex) to reset the base build to; empty = HEAD. Repo auto-deepened if the SHA is missing.

Default	(empty = HEAD)
Used when	BUILD_BOTH (optional)

Commit SHA (7 to 40 hex characters) the baseline build is reset to. Empty uses HEAD. If the commit is missing locally, the repository is deepened automatically.

CHOOSE_BUILD_CONFIG

choice

CHOOSE_BUILD_CONFIG

Distro config applied before packaging:
Velinux -> cp config.x86_64 .config && make olddefconfig
Anolis -> make anolis_defconfig
OpenEuler -> make openeuler_defconfig
OpenCloud -> make tencentconfig
others -> make <CUSTOM_CONFIG>
Debian/veLinux build-breaker symbols (cert keys, DEBUG_INFO_BTf, NET_VENDOR_NETRONOME) are auto-disabled after.

Velinux

Choices	Velinux, Anolis, OpenEuler, OpenCloud, others
Used when	BUILD_BOTH

Choice	Kernel config applied
Velinux	cp config.x86_64 .config && make olddefconfig
Anolis	make anolis_defconfig
OpenEuler	make openeuler_defconfig
OpenCloud	make tencentconfig
others	make <CUSTOM_CONFIG>

On Debian/veLinux, symbols that break packaging (cert keys, DEBUG_INFO_BTf, NET_VENDOR_NETRONOME) are disabled automatically afterwards.

CUSTOM_CONFIG

string

CUSTOM_CONFIG

Make config target for CHOOSE_BUILD_CONFIG=others (e.g. x86_64_defconfig, allmodconfig). Ignored otherwise.

Default	(empty)
Used when	CHOOSE_BUILD_CONFIG=others (required)

Make config target, for example x86_64_defconfig or allmodconfig. Required when CHOOSE_BUILD_CONFIG=others: the build fails in *Validate parameters* if it is empty. Ignored for the other choices.

EXTRA_CONFIGS_ENABLE

text

EXTRA_CONFIGS_ENABLE

Optional. Configs to enable, one per line (.config format), on top of CHOOSE_BUILD_CONFIG. PATCH kernel only.

Accepted: CONFIG_FOO (=y), CONFIG_FOO=y|m|42|"string".

Default	(empty)
Used when	BUILD_BOTH, patched kernel only

Extra configs to enable on top of CHOOSE_BUILD_CONFIG, one per line. Accepted: CONFIG_FOO (=y) or CONFIG_FOO=y|m|42|"string".

EXTRA_CONFIGS_DISABLE

text

EXTRA_CONFIGS_DISABLE

Optional. Configs to disable, one per line, on top of CHOOSE_BUILD_CONFIG. PATCH kernel only.

Accepted: CONFIG_FOO or "# CONFIG_FOO is not set".

Default	(empty)
Used when	BUILD_BOTH, patched kernel only

Configs to disable, one per line. Accepted: CONFIG_FOO or # CONFIG_FOO is not set.

5.4 Manual kernels (only for RPM_KERNEL_BUILD=MANUAL_BOTH_NO_BUILD)

MANUAL_PATCH_KERNEL string

MANUAL_PATCH_KERNEL

Preinstalled patch kernel release as in /boot/vmlinuz-<THIS>. Used by MANUAL_BOTH_NO_BUILD; empty otherwise.

Default	(empty)
Used when	MANUAL_BOTH_NO_BUILD and the run uses the patched kernel

Kernel release of the preinstalled patched kernel, exactly as in /boot/vmlinuz-<release> (check with `ls /boot/vmlinuz-*`). Only required when the chosen LKP_RUN_OPTIONS tests the patched kernel.

MANUAL_BASE_KERNEL string

MANUAL_BASE_KERNEL

Preinstalled base kernel release as in /boot/vmlinuz-<THIS>. Used by MANUAL_BOTH_NO_BUILD; empty otherwise.

Default	(empty)
Used when	MANUAL_BOTH_NO_BUILD and the run uses the baseline kernel

Same as above, for the baseline kernel. For BOTH_* / ALL_RUNS the two names must be different.

5.5 Virtual machines

VM_CREATION_REQUIRED choice

VM_CREATION_REQUIRED

Spawn stress VMs (YES/NO).

YES spawns one guest per OS from golden images in /vms/jenkins_qcow2 (6 VMs, <OS>_stressVM) plus EXTRA_VMS cloned from the host OS. Golden images are read-only (guests are qcow2 overlays) and verified present at start.

NO

▼

Choices	NO, YES
Used when	Must be YES for every *_IN_VM and ALL_* option

YES spawns one VM per OS (6 VMs, named <OS>_stressVM) from the golden images in /vms/jenkins_qcow2/ , plus EXTRA_VMS . VMs are throw-away qcow2 overlays; the golden images are never modified.

EXTRA_VMS choice

EXTRA_VMS

Extra guests beyond the 6 per-OS VMs (total = 6 + x), cloned from the host-OS golden image and named <hostOS>_stressVM_e1..e<x> . Default 0.

0

▼

Choices	0 to 5 (default 0)
Used when	VM_CREATION_REQUIRED=YES

Extra VMs on top of the 6 per-OS VMs, cloned from the image matching the host OS and named <hostOS>_stressVM_e1 ... The host OS must be one of the six known distributions.

VM_LOGIN_CREDENTIALS_ID string

VM_LOGIN_CREDENTIALS_ID

Only when VM_CREATION_REQUIRED=YES: Jenkins "Username with password" credential ID used to SSH into the current-OS guest to install the kernel package.

vm-login

Default	vm-login
Used when	VM runs that install a kernel into the guest

ID of the Jenkins *Username with password* credential used to SSH into the VM whose OS matches the host, to copy and install the kernel package.

MANUAL_BASE_RPM_PATH string

MANUAL_BASE_RPM_PATH

Only when VM_CREATION_REQUIRED=YES and the kernel was NOT built by this pipe: path on the agent to the BASE kernel .rpm (RHEL-family host). Installed into the current-OS guest during a BASE VM run. If the kernel was built by this pipe earlier (BUILD_BOTH), find the .rpm in /vms/jenkins_rpm/ (the name contains "base_patch" or "base-patch"; only the latest BUILD_BOTH run is kept).

Used when	VM run, baseline kernel, RHEL-family host, kernel not built in this run
-----------	---

Path on the machine to the baseline kernel `.rpm` to install inside the matching VM. Kernels built earlier by this pipeline are kept in `/vms/jenkins_rpm/` (name contains `base_patch` / `base-patch`; only the latest BUILD_BOTH run is kept). If left empty, the VMs keep their default kernel.

MANUAL_BASE_DEB_PATH string

MANUAL_BASE_DEB_PATH

Only when VM_CREATION_REQUIRED=YES and the kernel was NOT built by this pipe: path on the agent to the BASE kernel .deb (Debian/veLinux host). Installed into the current-OS guest during a BASE VM run. If the kernel was built by this pipe earlier (BUILD_BOTH), find the .deb in /vms/jenkins_rpm/ (the name contains "base_patch" or "base-patch"; only the latest BUILD_BOTH run is kept).

Used when	VM run, baseline kernel, Debian/veLinux host, kernel not built in this run
-----------	--

Same as above for a .deb package.

MANUAL_PATCH_RPM_PATH string

MANUAL_PATCH_RPM_PATH

Only when VM_CREATION_REQUIRED=YES and the kernel was NOT built by this pipe: path on the agent to the PATCH kernel .rpm (RHEL-family host). Installed into the current-OS guest during a PATCH VM run. If the kernel was built by this pipe earlier (BUILD_BOTH), find the .rpm in /vms/jenkins_rpm/ (the name contains "with_patch" or "with-patch"; only the latest BUILD_BOTH run is kept).

Used when	VM run, patched kernel, RHEL-family host, kernel not built in this run
-----------	--

Path to the patched kernel .rpm. Earlier builds: /vms/jenkins_rpm/ (name contains with_patch / with-patch).

MANUAL_PATCH_DEB_PATH string

MANUAL_PATCH_DEB_PATH

Only when VM_CREATION_REQUIRED=YES and the kernel was NOT built by this pipe: path on the agent to the PATCH kernel .deb (Debian/veLinux host). Installed into the current-OS guest during a PATCH VM run. If the kernel was built by this pipe earlier (BUILD_BOTH), find the .deb in /vms/jenkins_rpm/ (the name contains "with_patch" or "with-patch"; only the latest BUILD_BOTH run is kept).

Used when	VM run, patched kernel, Debian/veLinux host, kernel not built in this run
-----------	---

Same as above for a `.deb` package.

5.6 Boot and LKP

ALREADY_BOOTED choice

ALREADY_BOOTED

Skip grubby/reboot if the SUT already runs the target kernel.
NO — always boot when required (default).
BASE_KERNEL — skip boot for BASE-target runs (verifies `uname -r = BASE_KERNEL` first).
PATCH_KERNEL — skip boot for PATCH-target runs. Neither applies to the PATCH half of BOTH_*/ALL_RUNS.

NO

Choices	NO (default), BASE_KERNEL, PATCH_KERNEL
---------	---

Skips the set-default + reboot when the machine is already running the target kernel (the pipeline still checks `uname -r` first). Does not apply to the second (patched) half of BOTH_* / ALL_RUNS.

LKP_RUN_OPTIONS

choice

LKP_RUN_OPTIONS

Boot/LKP after kernels resolve (ignored when RPM_KERNEL_BUILD=NULL):

BASE_KERNEL_AND_LKP / PATCH_KERNEL_LKP — boot target kernel + LKP on host.

BOTH_BASE_PATCH_LKP — LKP on BASE then PATCH (ends on PATCH).

*_NOLKP_IN_VM — boot target, run LKP in VMs from non-LKP qcow2, cleanup.

*_LKP_IN_VM — boot target, run LKP in VMs from LKP qcow2, cleanup.

BOTH_KERNELS_*_IN_VM — VM run on BASE then PATCH (self-contained per phase).

ALL_BASE / ALL_PATCH — that kernel: host LKP + both VM runs.

ALL_RUNS — BASE then PATCH: host LKP + both VM runs.

BASE_KERNEL_AND_LKP

Chooses which kernel(s) run and which test setups are measured. The full table is in section 4 (Run choices).

WORKLOAD_TYPE

choice

WORKLOAD_TYPE

LKP workload(s):

hackbench / ebizzy / unixbench — that one only (unixbench has no iterations param).

hackbench_and_ebizzy — both.

ALL — all three.

hackbench

Choices	hackbench (default), ebizzy, unixbench, hackbench_and_ebizzy, ALL
---------	---

LKP workload(s) to run in each LKP stage.

HACKBENCH_ITERATIONS

choice

HACKBENCH_ITERATIONS

hackbench iterations; patched into lkp-tests/jobs/hackbench.yaml before split-job.

4

Choices	4 (default), 8
---------	----------------

Written into `lkp-tests/jobs/hackbench.yaml` before the jobs are generated.

EBIZZY_ITERATIONS

choice

EBIZZY_ITERATIONS

ebizzy iterations (the `x` suffix is added); patched into lkp-tests/jobs/ebizzy.yaml before split-job.

25

▼

Choices	25 (default), 50, 75, 100
---------	---------------------------

Written into lkp-tests/jobs/ebizzy.yaml (the x suffix is added automatically).

6. Automatic checks performed by the pipeline

Check	Fails when	Stage
Agent label	<code>NODE_LABEL</code> is empty, no online agent has that label within 1 minute, or the agent has no IPv4 address	Resolve node IP from agent
Prerequisites	<code>PREREQUISITES_CONFIRMED</code> is not ticked	Validate parameters
Jira ticket	<code>JIRA_ID</code> is empty	Validate parameters
Kernel tree	<code>BUILD_BOTH</code> and <code>EXISTING_REPO_PATH</code> / <code>EXISTING_REPO_BRANCH</code> is empty, the path does not exist, or it is not a git checkout	Validate parameters
Baseline SHA	<code>BASE_HEAD_SHA_ID</code> is not 7 to 40 hex characters	Validate parameters
Custom config	<code>CHOOSE_BUILD_CONFIG=others</code> and <code>CUSTOM_CONFIG</code> is empty	Validate parameters
Excel template	<code>LKP_RESULT_TEMPLATE_PATH</code> is empty or does not exist	Validate parameters
Manual kernels	<code>MANUAL_BOTH_NO_BUILD</code> and the kernel needed by <code>LKP_RUN_OPTIONS</code> is empty or <code>/boot/vmlinuz-<name></code> does not exist	Validate parameters
Distinct kernels	A <code>BASE+PATCH</code> run where <code>MANUAL_BASE_KERNEL</code> equals <code>MANUAL_PATCH_KERNEL</code>	Validate parameters
VM option	An <code>*_IN_VM</code> / <code>ALL_*</code> option with <code>VM_CREATION_REQUIRED=NO</code>	Validate parameters
Golden images	Any of the 12 images is missing from <code>/vms/jenkins_qcow2/</code> (when <code>VM_CREATION_REQUIRED=YES</code>)	Validate parameters
Extra VMs	<code>EXTRA_VMS</code> > 0 but the host OS is not one of the six known distributions	Validate parameters
Boot selection	The saved default / one-time boot entry is not exactly the target kernel (a similar name such as 6.1.61 for 6.1.6 does not count)	Set default kernel
Agent return	The agent is not back within 1 hour after a reboot	Reboot and wait
Booted kernel	After reboot, <code>uname -r</code> is not the expected kernel	Verify kernel

Most mistakes are caught within the first minute or two, before any build, reboot or VM creation.

7. Paths used

Path (on the test machine)	Used for
/vms/jenkins_qcow2/	12 golden VM images (read-only) and the per-run VM overlay disks
/vms/jenkins_excel_template/lkp_result_template.xlsx	Excel template (default of LKP_RESULT_TEMPLATE_PATH)
/vms/jenkins_rpm/	Kernel packages built by the latest BUILD_BOTH run (emptied at the start of each BUILD_BOTH run)
/tests/jenkins/workspace/	Jenkins agent remote root; each job workspace under it holds Run_<build number>/ with the raw LKP results and a copy of the workbook
/home/amd/	Working directory (PIPELINE_BASE_DIR)
/home/amd/LKP_Automated/	LKP suite cloned from GitHub and built on first use
/home/amd/DEAE_<JIRA_ID>/	Excel result workbook(s)
/lkp/result/<workload>/	Live LKP output of the current run (cleared before each LKP run)
EXISTING_REPO_PATH	Kernel git tree used by BUILD_BOTH
/boot/vmlinuz-*, grub config	Kernels to boot, and the default / next-boot selection
/etc/motd, /etc/motd.lkp_backup	"LKP running" banner during the run, and the saved original motd
/tmp/kernel_test.rpm or .deb (inside the VM)	Kernel package copied into the matching VM before install

8. Where to find the results

8.1 Excel workbook (main result)

```
/home/amd/DEAE_<JIRA_ID>/DEAE_<JIRA_ID>_<OS>_<kernel major.minor>_<job name>.xlsx  
example: /home/amd/DEAE_5937/DEAE_5937_veLinux_6.6_<job name>.xlsx
```

- One workbook per JIRA / OS / kernel line / job. Later runs with the same values update the same file, so BASE and PATCH results end up side by side.
- Each LKP run fills one column: host C/D, VM-as-load E/F, LKP-in-VM G/H (BASE/PATCH).
- Rows: hackbench 7 to 14, ebizzy 15, unixbench 16 to 43. Values are elapsed seconds.
- The delta % columns and summary are calculated by Excel formulas when the file is opened.
- After every LKP run the updated workbook is also copied to `<job workspace>/Run_<build number>/` (overwriting the previous copy), so it can be downloaded from Jenkins: job → build number → *Workspaces*.

8.2 Raw LKP result archive (in the Jenkins job workspace)

```
<job workspace>/Run_<build number>/<RUN_NAME>/lkp_result/  
example: /tests/jenkins/workspace/workspace/<folder>/<job name>/Run_3/BASE_ONLY_LKP/lkp_result/
```

The job workspace is the agent's remote root plus `workspace/<Jenkins folder path>/<job name>`; folder names keep their spaces. Browse or download it from Jenkins: job → build number → *Workspaces*.

RUN_NAME	Setup
BASE_ONLY_LKP / PATCH_ONLY_LKP	A. Machine only
BASE_VM / PATCH_VM	B. Machine + VMs as extra load
BASE_VM_LKP / PATCH_VM_LKP	C. LKP inside the VMs

Each folder is a full copy of `/lkp/result` for that run (per-workload directories, `*.time` files and `result.sh`).

8.3 Other outputs

- **Built kernel packages** (BUILD_BOTH): `/vms/jenkins_rpm/`
- **Build log**: Jenkins → job → build number → *Console Output* (shows kernel names, stage results and the workbook path).

9. Troubleshooting

Symptom	Likely cause / fix
Build fails in <i>Validate parameters</i>	Read the error line in the console; it names the missing parameter or file (JIRA_ID, kernel name, golden image, ...).
"Missing golden qcow2 image(s)"	Copy the images to <code>/vms/jenkins_qcow2/</code> from 10.86.26.102 and run <code>chown qemu:qemu</code> (section 2.3).
"PREREQUISITES_CONFIRMED is not ticked"	Check the checklist in section 2, tick the box and re-run.
"No online Jenkins agent with label ... within 1 minute"	The agent is offline or <code>NODE_LABEL</code> is wrong: check <i>Manage Jenkins</i> → <i>Nodes</i> , bring the agent online and re-run.
"LKP_RESULT_TEMPLATE_PATH not found"	Copy the template to <code>/vms/jenkins_excel_template/</code> (checked in <i>Validate parameters</i>).
Build waits after a reboot (fails after 1 hour)	The agent did not reconnect: check that the agent runs as a service and that the new kernel boots (use the machine console / BMC). On veLinux the IP may have changed: update the node's host IP in Jenkins and relaunch the agent; the build continues once it is online.
VMs never get an IP	<code>libvirt default</code> network is not active; run <code>virsh net-start default</code> and check the netfilter REJECT support of the host kernel.
Delta % columns empty in Excel	Press <code>Ctrl+Alt+F9</code> in Excel to force a full recalculation, then save.
"LKP Regression is running" banner still shown after a run	Restore it by hand: <code>mv /etc/motd.lkp_backup /etc/motd</code> .